i need an ai model that could work as you have to find k aagr kisi model k api key mil jaey for that purpose so uskaa name kam and usage and cost sab mention krnaa agar koi model hugging face ya githum per hai or open soucerce hai or trained bhi hai and could be used for this purpose Jab user ke phone par auto-scroll chal raha ho aur screen par koi video (Instagram Reels, TikTok, YouTube) aa jaye, to app ko real-time mein detect karna hai ke video chal rahi hai aur jab tak video khatam na ho, scroll ko rok ke rakhna hai.

Aapke use case ke liye — **jab phone par auto-scroll chal raha ho aur screen par koi video (Reels, TikTok, YouTube Shorts) aa jaye, to real-time detect karke scroll rok dena jab tak video khatam na ho** — yeh ek **on-device real-time video playback detection** ka problem hai. Iske liye koi single "AI model" nahi hai jo directly yeh kaam kare, lekin kuch practical approaches hain jo aap adopt kar sakte hain.

Best Approach: Android Accessibility Service (No AI Model Needed)

Sabse reliable aur cost-effective solution **AI model nahi, balki Android Accessibility Service** use karna hai. Yeh approach already production apps mein use ho raha hai:

NoScroll App Example

- **App Name**: NoScroll (Play Store par available)
- **Kaise kaam karta hai**: Android Accessibility Service use karke detect karta hai jab user Reels, Shorts, ya TikTok feed par hai
- **Detection Method**: UI element identifiers (resource IDs) aur active app package name check karta hai
- **Cost**: Free (koi API key nahi chahiye)
- **Open Source**: Nahi, lekin similar approach open source projects mein available hai

Kaise Implement Karein:

1. **Accessibility Service** banayein jo:
    - Current active app detect kare (Instagram, TikTok, YouTube)
    - Screen par visible UI elements check kare (video player controls, progress bar, etc.)
    - Jab video player detect ho, auto-scroll disable kar de
2. **Sample Code Structure**:

kotlin

```
override fun onAccessibilityEvent(event: AccessibilityEvent) {
    val rootNode = rootInActiveWindow
    if (rootNode != null) {
        if (isVideoPlaying(rootNode)) {
            disableAutoScroll()
        }
    }
}
```

Agar AI Model Hi Use Karna Hai

Agar aap AI-based approach chahte hain (screen content analyze karke), to yeh options hain:

1. TensorFlow Lite Video Classification

- **Model**: MoViNet, VideoMAE (Hugging Face par available)
- **Platform**: Android (TFLite compatible)
- **Cost**: Free (open source)
- **Usage**: Screen frames capture karke classify karein ke "video content" hai ya "static feed"
- **Limitation**: Heavy computation, battery drain

2. YOLO-based Screen Analysis

- **Model**: YOLOv5, YOLOv8, RT-DETR
- **Hugging Face Links**:
    - ultralytics/yolov5 (object detection)
    - facebook/rt-detr (real-time detection)
- **Cost**: Free
- **Usage**: Screen par video player UI elements detect karein (play button, progress bar)
- **Deployment**: TFLite mein convert karke mobile par run karein

3. Google Cloud Video Intelligence API

- **API Name**: Video Intelligence Streaming API

- **Feature**: STREAMING_OBJECT_TRACKING
- **Cost**: ~$1.50 per 1000 frames (2025 pricing)
- **Usage**: Real-time video stream analyze karein
- **Limitation**: Cloud-based, latency, cost

Recommended Open Source Models (Hugging Face)

Model Name	Type	Size	Mobile Ready	Cost
HuggingFaceTB/SmolVLM2-256M-Video	Video Understanding	256M	Yes (TFLite)	Free
facebook/sam3	Video Segmentation	Large	No (needs GPU)	Free
google/movinet	Video Classification	Small	Yes	Free
ultralytics/yolov5	Object Detection	Variable	Yes (TFLite)	Free

Practical Implementation Steps

1. **Accessibility Service Approach (Recommended)**:
    - No AI model needed
    - No API cost
    - Real-time, low latency
    - Already proven (NoScroll app)
2. **Hybrid Approach**:
    - Accessibility Service se app detect karein
    - Simple computer vision (OpenCV) se screen par video player UI detect karein
    - Koi heavy AI model nahi chahiye

3. **Pure AI Approach (Not Recommended)**:
    - Screen frames capture karein (every 500ms)
    - TFLite model se classify karein
    - High battery consumption, complex implementation

Cost Summary

Approach	API Key Needed	Monthly Cost	Complexity
Accessibility Service	❌ No	$0	Medium
TFLite (on-device)	❌ No	$0	High
Google Cloud Video API	✅ Yes	~$50-100+	Low
Hugging Face Models	❌ No	$0	High

Final Recommendation

Aapke use case ke liye **Accessibility Service** best hai — koi AI model, API key, ya cost nahi chahiye. GitHub par open source examples available hain jo aap modify kar sakte hain. Agar phir bhi AI model use karna hai, to **SmolVLM2-256M-Video** ya **YOLOv5 TFLite** try karein, lekin yeh overkill hoga aapke simple detection task ke liye.

SmolVLM2 video understanding models on Hugging Face

SmolVLM2 Video Understanding Models — Complete Guide

SmolVLM2 Hugging Face ka latest lightweight multimodal model family hai jo specifically **video understanding** ke liye design kiya gaya hai. Yeh models mobile/edge devices par bhi run kar sakte hain.

Model Variants (3 Sizes)

Model	Parameters	GPU RAM (Video)	Video-MME Score	Best For
SmolVLM2-2.2B-Instruct	2.2B	5.2 GB	52.1	Best quality video tasks
SmolVLM2-500M-Video-Instruct	500M	1.8 GB	42.2	Mobile/edge deployment
SmolVLM2-256M-Video-Instruct	256M	1.38 GB	33.7	Experimental, ultra-light

Key Features

- **Native Video Processing**: Direct video frames process kar sakta hai (na ke single images)
- **Multi-modal Input**: Video + Image + Text ko interleave karke samajh sakta hai
- **Apache 2.0 License**: Commercial use ke liye free, koi restriction nahi
- **MLX Support**: Apple Silicon (iPhone/Mac) par native Swift/Python API
- **Transformers Ready**: Hugging Face transformers library se directly load ho jata hai

Your Use Case Ke Liye Suitability

❌ **Not Recommended for Real-time Video Detection**

SmolVLM2 aapke original use case (auto-scroll stop when video plays) ke liye **overkill** aur **impractical** hai:

1. **Heavy Computation**: 256M model bhi 1.38GB RAM mangta hai — mobile par background mein continuously run nahi kar sakta
2. **Latency**: Video inference ke liye multiple frames process karna parta hai — real-time detection (milliseconds) ke liye slow hai
3. **Wrong Task**: SmolVLM2 **video content samajhne** ke liye hai (captioning, Q&A), na ke **video playback detect** karne ke liye
4. **Battery Drain**: Continuous screen analysis se battery 2-3 ghante mein khatam ho jayegi

Correct Models for Your Use Case

Agar AI-based screen analysis hi karna hai, to yeh better options hain:

1. MobileNet + SSD (TensorFlow Lite)

- **Size**: ~5-10 MB
- **RAM**: <100 MB
- **Latency**: <50ms per frame
- **Use**: Screen par video player UI elements detect karein (play button, progress bar)

2. YOLOv5/v8 Nano (TFLite)

- **Hugging Face**: ultralytics/yolov5
- **Size**: 2-5 MB (nano version)
- **Use**: Real-time object detection for video player controls

3. MoViNet-A0 (Google)

- **Hugging Face**: google/movinet
- **Size**: ~3 MB
- **Task**: Video classification (video vs static content)
- **Mobile Ready**: TensorFlow Lite compatible

SmolVLM2 Kab Use Karein?

SmolVLM2 tab use karein jab aapko chahiye:

- ✅ Video ka **summary/caption** generate karna
- ✅ Video ke baare mein **questions answer** karna
- ✅ **Long-form video analysis** (highlights extraction)
- ✅ Server/cloud par batch processing (real-time nahi)

Quick Start Code (SmolVLM2-256M)

python

```
from transformers import AutoProcessor, AutoModelForImageTextToText

import torch
```

```
model_path = "HuggingFaceTB/SmolVLM2-256M-Video-Instruct"
processor = AutoProcessor.from_pretrained(model_path)
model = AutoModelForImageTextToText.from_pretrained(
    model_path,
    torch_dtype=torch.bfloat16,
    _attn_implementation="flash_attention_2"
).to("cuda")

messages = [{
    "role": "user",
    "content": [
        {"type": "video", "path": "video.mp4"},
        {"type": "text", "text": "Is this a social media reel/short video?"}
    ]
}]

inputs = processor.apply_chat_template(
    messages, add_generation_prompt=True,
    tokenize=True, return_dict=True, return_tensors="pt"
).to(model.device, dtype=torch.bfloat16)

generated_ids = model.generate(**inputs, max_new_tokens=64)
print(processor.batch_decode(generated_ids, skip_special_tokens=True)[0])
```

Final Recommendation

Aapke **auto-scroll + video detection** use case ke liye:

1. **Best**: Android Accessibility Service (koi AI model nahi, zero cost)

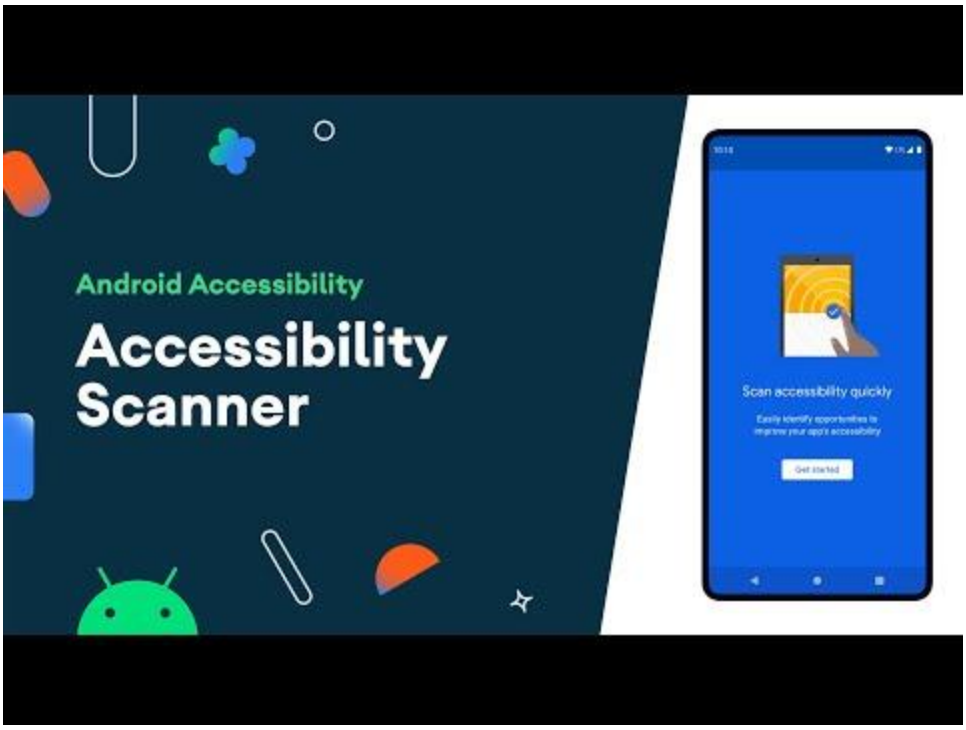
2. **Alternative**: YOLOv5 Nano TFLite (screen par video UI detect karein)
3. **Avoid**: SmolVLM2 (too heavy, wrong task, high latency)

SmolVLM2 excellent model hai, lekin aapke specific problem ke liye suitable nahi hai.

How to build a custom Android Accessibility Service YOLOv5 TFLite mobile deployment cost and performance

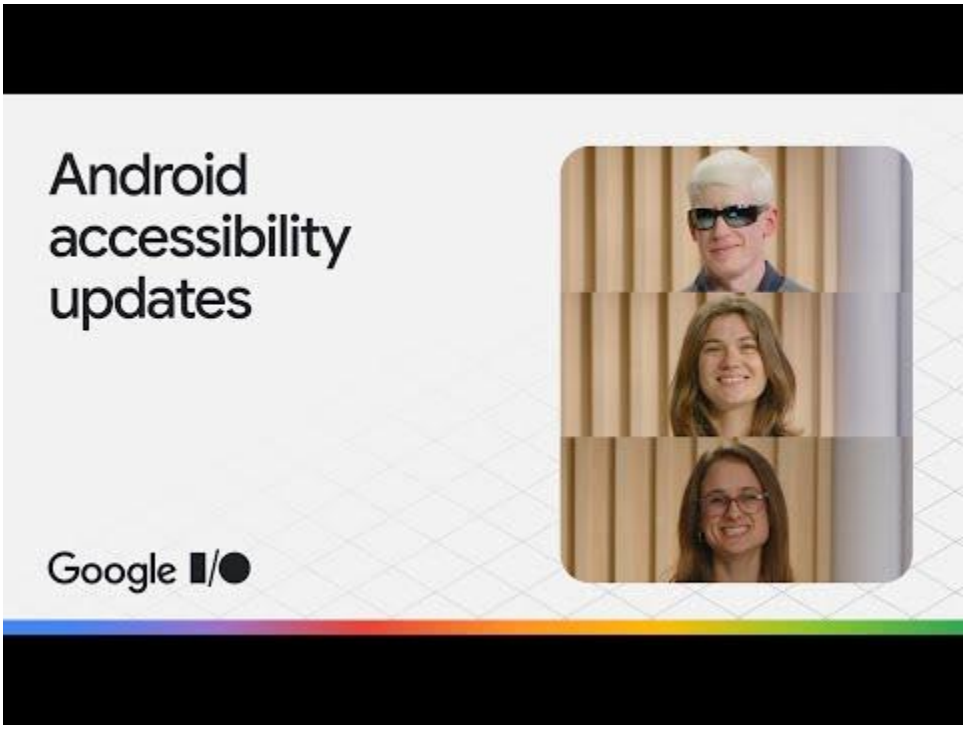
How to Build Screen Time Tracking App Using Android ...

1:05:03



Accessibility scanner - Accessibility on Android

4:55



Android accessibility updates

20:02

Complete Guide: Android Accessibility Service + YOLOv5 TFLite

Aapke use case ke liye **do approaches** hain — main dono ko detail mein explain karunga:

Part 1: Custom Android Accessibility Service (Recommended)

Step-by-Step Implementation

1. Project Setup

Android Studio mein new project banayein (Minimum SDK: API 24+)

2. Manifest Configuration (AndroidManifest.xml)

xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.videodetector">

    <application
        android:allowBackup="true"
        android:label="Video Detector">

        <service
            android:name=".VideoDetectionService"
            android:permission="android.permission.BIND_ACCESSIBILITY_SERVICE"
            android:exported="true">
            <intent-filter>
                <action android:name="android.accessibilityservice.AccessibilityService" />
            </intent-filter>
            <meta-data
                android:name="android.accessibilityservice"
```

```
            android:resource="@xml/accessibility_service_config" />
        </service>
    </application>
</manifest>
```

3. Service Configuration (res/xml/accessibility_service_config.xml)

xml

```
<accessibility-service xmlns:android="http://schemas.android.com/apk/res/android"
    android:description="@string/service_description"

android:packageNames="com.instagram.android,com.zhiliaoapp.musically,com.google.android.youtube"
    android:accessibilityEventTypes="typeWindowStateChanged|typeWindowContentChanged"
    android:accessibilityFlags="flagDefault|flagIncludeNotImportantViews"
    android:accessibilityFeedbackType="feedbackGeneric"
    android:notificationTimeout="100"
    android:canRetrieveWindowContent="true" />
```

4. Service Implementation (VideoDetectionService.kt)

kotlin

```
class VideoDetectionService : AccessibilityService() {

    override fun onAccessibilityEvent(event: AccessibilityEvent) {
        val rootNode = rootInActiveWindow ?: return

        try {
            if (isVideoPlaying(rootNode)) {
                disableAutoScroll()
            }
```

```
    } finally {
        rootNode.recycle()
    }
}

private fun isVideoPlaying(root: AccessibilityNodeInfo): Boolean {
    // Check for video player UI elements
    val videoIndicators = listOf(
        "video_player", "player_view", "exoplayer",
        "progress_bar", "play_button", "pause_button"
    )

    return findNodeByTextOrId(root, videoIndicators) != null
}

private fun findNodeByTextOrId(
    root: AccessibilityNodeInfo,
    indicators: List<String>
): AccessibilityNodeInfo? {
    for (i in 0 until root.childCount) {
        val child = root.getChild(i) ?: continue
        try {
            val viewId = child.viewIdResourceName ?: ""
            val contentDesc = child.contentDescription?.toString() ?: ""

            if (indicators.any { viewId.contains(it) || contentDesc.contains(it) }) {
```

```
                return child
            }

            // Recursive search
            val found = findNodeByTextOrId(child, indicators)
            if (found != null) return found
        } finally {
            child.recycle()
        }
    }
    return null
  }

  private fun disableAutoScroll() {
    // Send custom event to your auto-scroll service
    // Or use global action to pause scrolling
    performGlobalAction(GLOBAL_ACTION_MEDIA_PLAY_PAUSE)
  }

  override fun onInterrupt() {}
}
```

5. Enable Service in App

kotlin

```
// Settings screen se user ko enable karwana
val intent = Intent(Settings.ACTION_ACCESSIBILITY_SETTINGS)
startActivity(intent)
```

Open Source References

- **Blokky**: GitHub par open source (Reels/Shorts block karta hai)
- **Scrolless**: TikTok, Reels, Shorts detect aur block karta hai
- **Shorts Blocker**: YouTube Shorts auto-detect karke back button press karta hai

---

Part 2: YOLOv5 TFLite Mobile Deployment

Performance & Cost

Metric	**YOLOv5 Nano (320px)**	**YOLOv5 Small**	**YOLOv5 Medium**
Model Size	2-3 MB	14 MB	43 MB
Inference Time	6-18 ms	18-25 ms	22-56 ms
FPS	50-60 FPS	47-55 FPS	45 FPS
RAM Usage	6-43 MB	311 MB (peak)	74 MB
FLOPs	0.5B	15.8G	-

Device-Specific Performance (TFLite on NPU)

Device	**Chipset**	**Inference Time**	**RAM**
Samsung S24	Snapdragon 8 Gen 3	18.4 ms	6-74 MB
Samsung S23	Snapdragon 8 Gen 2	24.9 ms	6-43 MB
Mid-range	Snapdragon 8 Gen 1	25.4 ms	0-311 MB

Cost Summary

Component	Cost
YOLOv5 Model	Free (Apache 2.0)
TFLite Runtime	Free (built-in Android 10+)
Training	Free (Google Colab GPU)
API Key	Not required
Monthly Cost	$0

Implementation Steps

1. Download Pre-trained Model

bash

 # Hugging Face se TFLite model

wget https://huggingface.co/qualcomm/Yolo-v5/resolve/main/Yolo-v5.tflite

2. Add to Android Project

- app/src/main/assets/Yolo-v5.tflite folder mein copy karein
- build.gradle mein TFLite dependency add karein:

text

```
dependencies {
   implementation 'org.tensorflow:tensorflow-lite:2.14.0'
   implementation 'org.tensorflow:tensorflow-lite-support:0.4.4'
}
```

3. Detection Code

kotlin

```
class YoloDetector(context: Context) {
```

```
private val interpreter: Interpreter

init {
    val model = loadModelFile(context, "Yolo-v5.tflite")
    val options = Interpreter.Options()
    options.addDelegate(GpuDelegate()) // or NpuDelegate()
    interpreter = Interpreter(model, options)
}

fun detect(bitmap: Bitmap): List<Detection> {
    val input = preprocess(bitmap) // 320x320 resize
    val output = Array(1) { Array(1) { Array(25200) { FloatArray(85) } } }

    interpreter.run(input, output)
    return postProcess(output[0][0]) // NMS, confidence filter
}

private fun loadModelFile(context: Context, path: String): ByteBuffer {
    val fileDescriptor = context.assets.openFd(path)
    val inputStream = FileInputStream(fileDescriptor.fileDescriptor)
    val fileChannel = inputStream.channel
    val mapBuffer = fileChannel.map(
        FileChannel.MapMode.READ_ONLY,
        0,
        fileChannel.size()
    )
```

```
        return mapBuffer
    }
}
```

4. Screen Capture Integration

```
kotlin
// MediaProjection API se screen capture
val mediaProjection = mediaProjectionManager.createMediaProjection(
    resultCode, resultData
)
val imageReader = ImageReader.newInstance(width, height, format, 2)
```

Hybrid Solution (Best for Your Use Case)

Architecture

```
text
Accessibility Service (detect app)
    ↓
IF (Instagram/TikTok/YouTube detected)
    ↓
YOLOv5 TFLite (screen par video player UI detect karein)
    ↓
IF (video player detected)
    ↓
Disable auto-scroll
```

Why Hybrid?

1. **Battery Efficient**: Accessibility Service always on, YOLOv5 sirf jab needed
2. **Fast Detection**: <30ms total latency

3. **Zero Cost**: No API, no cloud
4. **Proven**: Blokky, Scrolless jaise apps already use kar rahe hain

---

GitHub Repos for Reference

1. **Blokky** (Reels/Shorts blocker): https://github.com/Ronjar/Blokky
2. **Scrolless** (Multi-platform): https://github.com/DuarteBarbosaDev/Scrolless
3. **Shorts Blocker**: https://github.com/atick-faisal/Shorts-Blocker
4. **YOLOv5 Android**: https://github.com/vokhidovhusan/yolov5-android
5. **MobileNet-YOLO** (3MB model): https://github.com/dog-qiuqiu/MobileNet-Yolo

---

Final Recommendation

Aapke liye **Accessibility Service + YOLOv5 Nano TFLite** best combination hai:

- **Total Cost**: $0
- **Development Time**: 2-3 days
- **Performance**: 30-50 FPS on mid-range phones
- **Battery**: Minimal impact (service background mein lightweight)

Start with **Accessibility Service** (Blokky code fork karke), phir gradually YOLOv5 add karein for better accuracy.

How to capture screen frames for YOLOv5 in an Accessibility Service Optimizing YOLOv5 TFLite inference speed on mobile NPUs Handling Android background service limitations with TFLite models Comparing YOLOv5 and YOLOv8 performance on Android TFLite Reducing YOLOv5 model size with quantization for Android deployment

How to pass camera frames from Flutter to TensorFlow Lite

Optimizing YOLOv5 performance with INT8 quantization in Flutter

Handling post-processing bounding box scaling for letterboxed images

Setting up RenderScript Intrinsics Replacement Toolkit on Android

Using Vulkan delegate instead of NNAPI for YOLOv5 in Flutter